What is SQLite Database?

SQLite is a **lightweight, file-based database**.

- It is **serverless** → No need to install a database server like MySQL
- It stores data in a **single file (.db / .sqlite)**
- It uses **SQL (Structured Query Language)**

In simple words:

SQLite is a **mini database inside a file**

Why Use SQLite?

Advantages:

- No server required
- Very fast for small apps
- Easy to use
- Works offline
- Zero configuration
- Portable (just copy file)

Why Use SQLite in Flutter?

Flutter apps need to store data locally like:

- User login info
- Notes / To-do list
- Offline app data
- Cache data

SQLite is perfect because:

- Works **offline**
- Fast performance
- Supported via plugin: sqflite

How to Download SQLite Software

You should first practice SQLite outside Flutter.

Option 1: SQLite Browser

Use: DB Browser for SQLite

Steps:

1. Go to official website

2. Download for Windows
3. Install normally

SQLite Basic Concepts

1. Database

A file that stores data
Example: `student.db`

2. Table

Collection of rows & columns
Example:

id	name	age

3. Row (Record)

Single entry
Example: `1, Neeraj, 23`

4. Column

Field of data
Example: `name`, `age`

Create Table in SQLites

1. Create Table

Example: **Employee Table**

```
CREATE TABLE employees (
    emp_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    department TEXT,
    salary REAL,
    age INTEGER,
    city TEXT
);
```

2. Insert Data (Single Row)

```
INSERT INTO employees (name, department, salary, age, city)
VALUES ('Neeraj', 'IT', 45000, 23, 'Lucknow');
```

3. Insert Multiple Data (Important)

```
INSERT INTO employees (name, department, salary, age, city) VALUES
('Amit', 'HR', 30000, 25, 'Delhi'),
```

```
('Riya', 'Finance', 40000, 24, 'Mumbai'),
('Sohan', 'IT', 50000, 26, 'Pune'),
('Priya', 'Marketing', 35000, 22, 'Noida'),
('Rahul', 'IT', 60000, 27, 'Bangalore');
```

4. Read Data (SELECT Queries)

All Data

```
SELECT * FROM employees;
```

Specific Columns

```
SELECT name, salary FROM employees;
```

With Condition (WHERE)

```
SELECT * FROM employees
WHERE department = 'IT';
```

With Multiple Conditions

```
SELECT * FROM employees
WHERE salary > 40000 AND city = 'Pune';
```

5. Update Data

Update Single Record

```
UPDATE employees
SET salary = 55000
WHERE emp_id = 3;
```

Update Multiple Conditions

```
UPDATE employees
SET city = 'Hyderabad'
WHERE department = 'IT';
```

6. Delete Data

Delete One Record

```
DELETE FROM employees
WHERE emp_id = 2;
```

Delete Multiple Records

```
DELETE FROM employees
WHERE department = 'HR';
```

Important Notes

- Always use **WHERE** in UPDATE & DELETE
  Otherwise all data will be affected

```
-- Dangerous (Don't do this)
DELETE FROM employees;
```

Practice Tasks (Do This Yourself)

1. Create table: `students`
2. Columns:
   - id, name, course, marks, city
3. Insert 5 records
4. Show students with marks > 70
5. Update marks of one student
6. Delete one student

SQLite Data Types

SQLite uses **dynamic typing** (very important concept).

Means:

You can store any type of value in any column (but still recommended to follow proper types)

Main Storage Classes in SQLite

SQLite has only **5 main data types**:

1. INTEGER

- Whole numbers
- Example: `10, -5, 1000`

```
age INTEGER
```

2. REAL

- Decimal / floating numbers
- Example: `10.5, 99.99`

```
salary REAL
```

3. TEXT

- String / characters
- Example: `'Neeraj', 'Hello'`

```
name TEXT
```

4. BLOB

- Binary data (images, files)

```
file_data BLOB
```

5. NULL

- Empty / no value

```
middle_name TEXT NULL
```

Type Affinity (Important Concept)

Even if you write:

```
price VARCHAR(20)
```

SQLite treats it as:

TEXT

Common Mappings:

Declared Type	SQLite Type
INT	INTEGER
VARCHAR	TEXT
CHAR	TEXT
FLOAT	REAL
DOUBLE	REAL

SQLite Constraints

Constraints are rules applied to columns.

Used to **control data and maintain accuracy**

1. PRIMARY KEY

- Unique + Not Null
- Identifies each row

```
id INTEGER PRIMARY KEY
```

2. AUTOINCREMENT

- Automatically increases value

```
id INTEGER PRIMARY KEY AUTOINCREMENT
```

3. NOT NULL

- Value cannot be empty

```
name TEXT NOT NULL
```

4. UNIQUE

- No duplicate values allowed

```
email TEXT UNIQUE
```

5. DEFAULT

- Sets default value

```
city TEXT DEFAULT 'Lucknow'
```

6. CHECK

- Condition must be true

```
age INTEGER CHECK(age >= 18)
```

7. FOREIGN KEY

- Links two tables

```
dept_id INTEGER,
FOREIGN KEY (dept_id) REFERENCES department(id)
```

Full Example (Table with Constraints)

```
CREATE TABLE employees (
    emp_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE,
    age INTEGER CHECK(age >= 18),
    salary REAL DEFAULT 20000,
    city TEXT DEFAULT 'Lucknow'
);
```

Insert Example (Constraint Working)

```
INSERT INTO employees (name, email, age)
VALUES ('Neeraj', 'neeraj@gmail.com', 23);
```

salary = 20000 (default)
city = Lucknow (default)

Constraint Violation Example

```
INSERT INTO employees (name, age)
VALUES (NULL, 20);
```

Error because `name` is NOT NULL

Important Points

- SQLite is flexible but constraints make it **safe**
- Always use:
  - PRIMARY KEY
  - NOT NULL
  - UNIQUE (where needed)
- CHECK is very useful in real apps

Logical Operators in SQLite

1. AND Operator

Used to combine multiple conditions
All conditions must be TRUE

```
SELECT * FROM employees
WHERE department = 'IT' AND salary > 40000;
```

Meaning:
Employees who are in IT **AND** salary > 40000

2. OR Operator

At least one condition must be TRUE

```
SELECT * FROM employees
WHERE department = 'HR' OR city = 'Delhi';
```

Meaning:
Employees in HR **OR** from Delhi

3. NOT Operator

Reverses condition

```
SELECT * FROM employees
WHERE NOT department = 'IT';
```

Meaning:
All employees **except IT**

4. IN Operator

Used to match multiple values

```
SELECT * FROM employees
WHERE city IN ('Delhi', 'Mumbai', 'Pune');
```

Meaning:
Employees from any of these cities

5. LIKE Operator

Used for pattern matching (search)

Starts with

```
SELECT * FROM employees
WHERE name LIKE 'A%';
```

Names starting with A

Ends with

```
SELECT * FROM employees
WHERE name LIKE '%a';
```

Names ending with a

Contains

```
SELECT * FROM employees
WHERE name LIKE '%ee%';
```

Names containing "ee"

6. BETWEEN Operator

Used for range

```
SELECT * FROM employees
WHERE salary BETWEEN 30000 AND 50000;
```

Salary between 30k and 50k

7. NOT BETWEEN

Opposite of BETWEEN

```
SELECT * FROM employees
WHERE age NOT BETWEEN 22 AND 25;
```

Age outside 22–25

Combined Example (Very Important)

```
SELECT * FROM employees
WHERE department = 'IT'
AND salary BETWEEN 40000 AND 60000
AND city IN ('Pune', 'Bangalore');
```

Real-life filter:

- IT employees
- Salary between 40k–60k
- Location Pune/Bangalore

Important Notes

- `'A%'` → starts with
- `'%A'` → ends with
- `'%A%'` → contains
- BETWEEN is **inclusive** (includes both values)

Practice Tasks

Try in SQLite:

1. Employees with age > 25 AND salary > 40000
2. Employees from Delhi OR Mumbai
3. Names starting with 'R'
4. Salary between 20000 and 50000
5. Employees NOT in IT department

Some More Operators

1. IS NULL

Used to find **missing / empty values**

```
SELECT * FROM employees
WHERE city IS NULL;
```

Meaning:
Employees whose **city is not filled**

2. IS NOT NULL

Used to find **non-empty values**

```
SELECT * FROM employees
WHERE salary IS NOT NULL;
```

Meaning:
Employees whose **salary is available**

Important Rule

Wrong:

```
WHERE city = NULL
```

Correct:

```
WHERE city IS NULL
```

Always use **IS NULL / IS NOT NULL**, not `= NULL`

3. LIMIT Clause

Used to **restrict number of rows**

```
SELECT * FROM employees
LIMIT 3;
```

Meaning:
Show only **first 3 records**

4. LIMIT with ORDER BY (Important)

```
SELECT * FROM employees
ORDER BY salary DESC
LIMIT 2;
```

Meaning:
Top 2 highest salary employees

5. OFFSET

Used to **skip rows**

```
SELECT * FROM employees
LIMIT 3 OFFSET 2;
```

Meaning:

- Skip first 2 rows
- Then show next 3 rows

Alternative Syntax (SQLite supports this)

```
SELECT * FROM employees
LIMIT 2, 3;
```

Meaning:

- Skip 2 rows

- Show next 3 rows

Same as:

```
LIMIT 3 OFFSET 2
```

Real-Life Example (Pagination)

```
SELECT * FROM employees
LIMIT 5 OFFSET 0;   -- Page 1

SELECT * FROM employees
LIMIT 5 OFFSET 5;   -- Page 2

SELECT * FROM employees
LIMIT 5 OFFSET 10;  -- Page 3
```

Used in apps (Flutter, websites)

Combined Example

```
SELECT * FROM employees
WHERE city IS NOT NULL
ORDER BY age DESC
LIMIT 3 OFFSET 1;
```

Meaning:

- Only records with city
- Sorted by age (highest first)
- Skip 1 record
- Show next 3

Important Notes

- LIMIT controls **how many rows**
- OFFSET controls **from where to start**
- Always use ORDER BY with LIMIT for correct results

Practice Tasks

1. Find employees where salary is NULL
2. Show top 3 youngest employees
3. Skip first 2 records and show next 4
4. Show employees where city is NOT NULL

Aggregate Functions in SQLite

Aggregate functions are used to **perform calculations on multiple rows** and return **one result**.

Common Aggregate Functions

1. COUNT()

Counts number of rows

```
SELECT COUNT(*) FROM employees;
```

Total number of employees

2. SUM()

Adds values

```
SELECT SUM(salary) FROM employees;
```

Total salary of all employees

3. AVG()

Calculates average

```
SELECT AVG(salary) FROM employees;
```

Average salary

4. MAX()

Highest value

```
SELECT MAX(salary) FROM employees;
```

Highest salary

5. MIN()

Lowest value

```
SELECT MIN(age) FROM employees;
```

Youngest employee age

Aggregate with GROUP BY

Used to group data

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

Average salary per department

Aggregate with HAVING

Used to filter grouped data

```
SELECT department, AVG(salary) as avg_salary
FROM employees
GROUP BY department
HAVING avg_salary > 40000;
```

Departments with avg salary > 40000

PRIMARY KEY + FOREIGN KEY

Used to **connect tables (relations)**

Step 1: Create Parent Table

```
CREATE TABLE departments (
    dept_id INTEGER PRIMARY KEY AUTOINCREMENT,
    dept_name TEXT NOT NULL
);
```

Step 2: Create Child Table (Foreign Key)

```
CREATE TABLE employees (
    emp_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    salary REAL,
    dept_id INTEGER,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

Concept

- **Primary Key** → Unique identifier (in parent table)
- **Foreign Key** → Refers to primary key of another table

Example:

- departments.dept_id = Primary Key
- employees.dept_id = Foreign Key

Insert Data

Insert into departments

```
INSERT INTO departments (dept_name)
VALUES ('IT'), ('HR'), ('Finance');
```

Insert into employees

```
INSERT INTO employees (name, salary, dept_id)
VALUES
('Neeraj', 50000, 1),
('Amit', 30000, 2),
('Riya', 40000, 3);
```

dept_id must exist in departments table

Use Both Tables (JOIN)

To fetch related data

```
SELECT employees.name, employees.salary, departments.dept_name
FROM employees
JOIN departments
ON employees.dept_id = departments.dept_id;
```

Output:

- Employee name
- Salary
- Department name

Important Notes

- Foreign key ensures **data integrity**
- You cannot insert invalid dept_id (if constraint is active)
- Always design database using relations in real projects

Practice Tasks

1. Create table `courses`
2. Create table `students` with course_id (foreign key)
3. Insert data in both tables
4. Show student name + course name using JOIN
5. Find average marks using AVG()